

Fountain Engine: A User Guide

Shenghao Yang

May 29, 2026

About this Document

This document introduces the Fountain Engine, a generic framework for fountain codes. The engine enables flexible implementation of various fountain code designs through a code scheme interface, and it is independent of any specific data operation implementation. The encoder and decoder provided by the engine generate the operations that need to be performed on data using an abstract data operation format. Users of the engine have complete freedom and control over how to carry out these operations. This document describes how to use the engine, including details on the code scheme interface, the data operation format, and the data operator interface. A companion Fountain Utility library is also introduced for code scheme validation and performance analysis.

This document applies to `fountain_engine` 1.2.0, and `fountain_utility` 1.1.0.

Contents

1	Introduction	3
2	Basic Usage	5
2.1	Encoding	5
2.2	Decoding	6
3	Coding Scheme	7
3.1	Fountain Code Model	9

3.2	Code Scheme Interface	10
3.2.1	Coding Parameters	10
3.2.2	Code Type	11
3.2.3	Degree Set Generator	11
3.2.4	Precode Generator	12
3.2.5	Decoding Configuration	12
3.3	Precode Traits	12
3.3.1	LDPC Precode	12
3.3.2	HDPC Precode	13
4	Data Manager	15
4.1	Types of Vectors	16
4.1.1	Message Vectors	16
4.1.2	Variable Vectors	16
4.1.3	Coded Vectors	17
4.2	Usage of Coded Vector IDs	17
4.2.1	Encoding	17
4.2.2	Decoding	18
4.3	Management of Data IDs	18
4.4	Finite Field Configuration	19
4.5	Data Operation Interfaces	19
4.5.1	Data Vector Operations	20
4.5.2	Informative Operation	21
4.5.3	Extract Operations	21
5	Data Operators	22
5.1	Operation Enum	22
5.2	Delayed Data Operation	22
5.2.1	Encoding Operation	23
5.2.2	Decoding Operation	24
5.2.3	Optimization of Data Operation Execution	26
5.3	On-the-fly Data Operation Retrieval	26
5.3.1	Encoder and Decoder with Data Operator	27
5.3.2	Display Data Operator	28
5.4	On-the-fly Data Operation Execution	29

6	Fountain Utility	30
6.1	Data Operators	30
6.1.1	IoDataOperator	30
6.1.2	VecDataOperater	31
6.2	Performance Metrics	32
6.2.1	PerformanceMetrics Struct	32
6.2.2	Computational Costs	33
6.2.3	Decoding Overhead	34
6.2.4	Storage Costs	34
6.3	Code Testing	35
6.3.1	Testing Templates	35
6.3.2	Testing Statistics	37

1 Introduction

Fountain codes are a class of erasure codes capable of generating a theoretically unlimited number of coded vectors from a given set of message vectors. Both LT codes and Raptor codes are subclasses of fountain codes [1, 2]. Raptor10 (R10) [3] and RaptorQ (RQ) [4] are two standardized Raptor codes by the IETF that demonstrate various techniques used in fountain code design.

Fountain codes are well-suited for network communications, where they provide reliability without requiring feedback mechanisms, flexible erasure correction capability, and low computational complexity. They have been adopted in several practical applications: 3GPP and DVB-H utilize Raptor10 for mobile wireless broadcasting and multicasting; the ANDL protocol employs a Reliable Large Datagram Protocol (RLDP) [5] based on RaptorQ [4]; and the Libreicast Project [6] implements IPv6 multicast using RaptorQ. Raptor codes have been implemented in numerous open source projects across multiple languages, including C++ [7], Java [8], and Rust [9, 10].

This document introduces *fountain engine*, henceforth referred to as *the engine*, a generic implementation framework for fountain code encoding and decoding. The engine is different from the existing implementations of fountain code mentioned above due to its generic nature, which is elaborated in two aspects.

The first generic aspect of the engine is that it does not implement any specific fountain code scheme. Instead, it provides a framework that can

encode and decode any fountain code specified through the code scheme interface. In contrast to the existing implementations of fountain codes that support only one specific fountain code design, the engine is much more flexible and can be used to implement various fountain code designs. This design benefits from the common encoding and decoding algorithm of fountain codes.

The second generic aspect of the engine is its independence from any specific data operation implementation. Fountain code encoding and decoding involve various data operations, including data padding, blocking, and vector operations such as scalar-vector multiplication and vector addition. The engine does not perform these data operations, but only specifies the operations that need to be performed on data vectors, assuming that the original data has already been properly padded and separated into equal-length data vectors. The encoding and decoding algorithms are vector operations performed on the data vectors, such as “add vector x to vector y ” or “multiply vector z by a scalar” based on vector IDs “ x, y, z ”. Users of the engine can freely use their own data operator to perform the operations.

The overall framework of the engine is shown in Figure 1. The `fountain_engine` includes the encoding and decoding algorithms based on generic code scheme interface and data operation interface. The `fountain_scheme` library provides some code scheme components and example code schemes, and the `fountain_utility` library provides some example data operators and performance analysis tools. In this document, we introduce the use of the engine and the utility library.

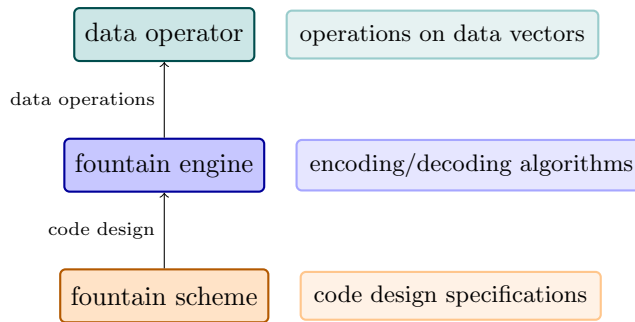


Figure 1: Overall framework of `fountain_engine` includes the code scheme, the engine, and the data operator.

2 Basic Usage

The `fountain_engine` is implemented as a Rust library. To use the library, you need to add it as a dependency to your Rust project.

```

1 [dependencies]
2 fountain_engine = "1.0"
3 fountain_utility = "1.0"
4 fountain_scheme = "1.0"

```

Remember to replace 1.0 with the version numbers you want to use.

In this section, we introduce a basic use case of the engine, which does not need a data operator. This use case can help to verify the correctness of a code scheme and evaluate the computation and memory cost by analyzing the operations generated by the engine using the `fountain_utility` library to be introduced in Section 6.

2.1 Encoding

The encoding process is implemented by the `Encoder` struct. To create an encoder, call the `new` function, which takes a coding scheme as argument.

```

1 use fountain_engine::Encoder;
2 use fountain_scheme::RandomLTCode;
3 use fountain_scheme::validation::pseudo_rand::XorShift64;
4 let rng = XorShift64::new(0xFEED);
5 let code_scheme = RandomLTCode::new_from_ideal_soliton(k, rng);
6 let encoder = Encoder::new(code_scheme);

```

The `code_scheme` is an implementation of the `CodeScheme` trait, which defines the interface for a code scheme. Here we use a random LT code scheme provided by the `fountain_scheme` library, where `XorShift64` is a pseudo-random number generator provided by the `fountain_scheme` library. We will discuss the code scheme in Section 3.

After the encoder is created, the encoding process can be started by calling the `encode_coded_vector` function. The function can be called multiple times for different coded vector IDs. The function returns `Option<usize>`:

if the `coded_id` is valid, `Some(data_id)` is returned, where `data_id` is the ID of the data vector for the coded vector; otherwise `None` is returned. The valid range of coded vector IDs will be discussed in Section 4.

```

1 if let Some(data_id) = encoder.encode_coded_vector(coded_id) {
2     // Use the data_id
3 }

```

After performing encoding, the operations generated by the encoder can be obtained by calling the `get_operations` function of the `manager` field. The operations can be executed by a data operator to generate the coded vectors. See Section 5 for the details about the data operator.

```

1 let operations = encoder.manager.get_operations();

```

2.2 Decoding

The decoding process is implemented by the `Decoder` struct. To create a decoder, call the `new` function, which takes a coding scheme as argument. The `code_scheme` should be the same as the one used to create the corresponding encoder.

```

1 use fountain_engine::Decoder;
2 use fountain_scheme::RandomLTCode;
3 use fountain_scheme::validation::pseudo_rand::XorShift64;
4 let rng = XorShift64::new(0xFEED);
5 let code_scheme = RandomLTCode::new_from_ideal_soliton(k, rng);
6 let decoder = Decoder::new(code_scheme);

```

After the decoder is created, the decoding process can be started by calling either `add_coded_id` or `add_coded_vector` function, both of which return the decoding status.

The `add_coded_id` function is used for delayed execution, where the coded vector data is not provided immediately. The coded vector data will be provided later through the operations generated by the decoder.

```
1 let status = decoder.add_coded_id(coded_id);
```

The `add_coded_vector` function is used for on-the-fly execution, where the coded vector data is provided immediately.

```
1 let status = decoder.add_coded_vector(coded_id, vector);
```

The `coded_id` is the ID of the coded vector to be decoded, and it should be the same as the one used to encode the coded vector. The `status` is a `DecodeStatus` enum, which can be `Decoded` or `NotDecoded`. If `NotDecoded`, more coded vectors can be added to the decoder.

The current decoding status can also be checked without adding a new coded vector by calling the `decode_status` function:

```
1 let status = decoder.decode_status();
```

The operations generated by the decoder can be obtained by calling the `get_operations` function of the `manager` field. The operations can be executed by a data operator to generate the decoded message vectors.

```
1 let operations = decoder.manager.get_operations();
```

3 Coding Scheme

A fountain code scheme consists of the following components: code parameters, a code type, a degree set generator, a precode generator, and a decoding configuration. These components should be implemented following the `CodeScheme` trait. Listing 1 shows an example of a code scheme implementation. Using this code scheme implementation, a code scheme for k message vectors can be created by calling the `new` function.

The code scheme shown in Listing 1 is an illustrative example. For research and development purposes, consider the code schemes available in the `fountain_scheme` library (crates.io). Additionally, specialized schemes are provided in the `fountain_raptor_q` and `fountain_raptor_10` libraries

```

1 use fountain_engine::traits::{CodeScheme, HDPC, LDPC};
2 use fountain_engine::types::{CodeParams, CodeType};
3
4 struct MyCodeScheme {
5     k: usize,
6 }
7
8 impl MyCodeScheme {
9     fn new(k: usize) -> Self {
10         Self { k }
11     }
12 }
13
14 impl CodeScheme for MyCodeScheme {
15     fn get_params(&self) -> CodeParams {
16         CodeParams::new(self.k, self.k, 0, 0)
17     }
18     fn code_type(&self) -> CodeType {
19         CodeType::Ordinary
20     }
21     fn create_degree_set_fn(&self) -> DegreeSetFn {
22         let k = self.k;
23         Box::new(move |coded_id| (0..k).collect:::<Vec<usize>>())
24     }
25     fn create_precode(&self) ->
26     (Option<Box<dyn HDPC>>, Option<Box<dyn LDPC>>) {
27         (None, None)
28     }
29 }

```

Listing 1: Example of a code scheme implementation

(crates.io and crates.io), implementing standards-compliant RaptorQ and Raptor10 codes, respectively.

In the remainder of this section, we first describe the general model of the engine used for encoding and decoding, and then discuss the components of a code scheme.

3.1 Fountain Code Model

The engine performs the fountain code encoding and decoding by processing vectors over a vector space over $\text{GF}(256)$, each of which is represented by a sequence of 8-bit integers. The vector length does not necessarily need to be known by the engine, but is assumed to be the same for all the vectors.

Suppose that the encoder operates on k message vectors, where k is a positive integer. The encoding has two steps: precoding and LT encoding. The precoding generates $k + l + h$ variable vectors by linear combinations of the message vectors, where l vectors are LDPC vectors, h vectors are HDPC vectors, and the remaining k vectors are called base vectors. The base vectors are the message vectors for the systematic encoding, but not for the ordinary encoding.

The LT encoding generates coded vectors by linear combinations of the variable vectors. Each coded vector is the sum of only a small subset of the variable vectors, which is called the degree set of the coded vector. The degree set of a coded vector is required for both encoding and decoding.

The overall encoding process is shown in Figure 2. Both the LDPC and HDPC precodes can be optional and customized by the code scheme. The LDPC precode and the LT encoding are performed over $\text{GF}(2)$. An HDPC precode may be **binary** or **$\text{GF}(256)$** .

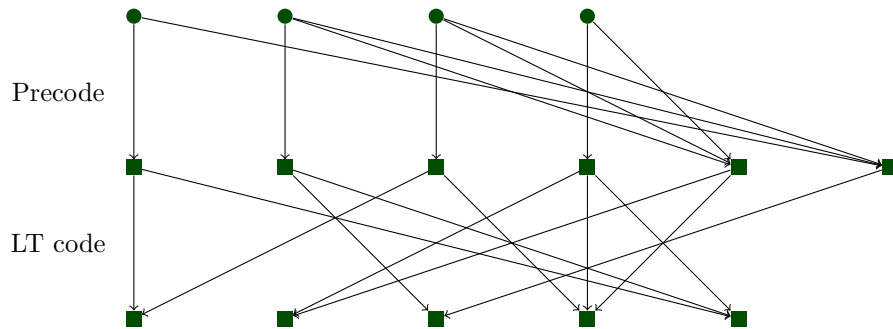


Figure 2: Overall encoding process of `fountain_engine`

For encoding, the k message vectors are provided as input, where the last p vectors are padding vectors. The padding vectors are all-zero vectors. For decoding, only the first $k - p$ message vectors are recovered.

3.2 Code Scheme Interface

The code scheme interface is defined by the `CodeScheme` trait. The trait defines the following functions:

- `get_params`: generate the coding parameters,
- `code_type`: specify the code type,
- `create_degree_set_fn`: create the degree set generator,
- `create_precode`: create the precode, and
- `decoding_config`: create the decoding configuration.

Here we only introduce the code scheme interface briefly for understanding the engine. Readers can refer to Section 3 for the details of the code scheme implementation.

3.2.1 Coding Parameters

A code scheme can generate the coding parameters by the `get_params` function, which returns a `CodeParams` struct. The `CodeParams` struct stores the following parameters:

- k : number of message vectors,
- a : number of active message vectors,
- l : number of LDPC vectors,
- h : number of HDPC vectors,
- $b = k - a$: number of inactive message vectors, and
- i : number of pre-inactive variable vectors (how many variable nodes are marked pre-inactive in the decoder graph for this parameter set).

Two functions are provided to create the `CodeParams` struct:

- `CodeParams::new(k, a, l, h)` sets $b = k - a$ and $i = b + h$ (pre-inactivation encoded in the struct).
- `CodeParams::new_without_pre_inact(k, l, h)` sets $a = k$, $b = 0$, and $i = 0$ (no pre-inactivation; e.g. Raptor10-style parameters).

When implementing `get_params`, use the constructor that matches the scheme. For example:

```
1 let params = CodeParams::new(k, a, l, h);
```

```
1 let params = CodeParams::new_without_pre_inact(k, l, h);
```

3.2.2 Code Type

The `code_type` function returns a `CodeType` enum.

```
1 pub enum CodeType {
2     Ordinary,
3     Systematic,
4 }
```

`Ordinary` and `Systematic` are used for the ordinary fountain code and the systematic fountain code, respectively. The ordinary fountain code does not include the message vectors in the coded vectors, while the systematic fountain code includes the message vectors in the coded vectors.

3.2.3 Degree Set Generator

The `create_degree_set_fn` function returns a function `degree_set` of type `DegreeSetFn`.

```
1 type DegreeSetFn = Box<dyn FnMut(usize) -> Vec<usize>>;
```

The function `degree_set` generates the degree set for a given coded vector ID. The function returns a list of the indices of the variable vectors with distinct values from 0 to $k + l + h - 1$.

3.2.4 Precode Generator

The precode generator is a function that generates the precode, which consists of an LDPC code and an HDPC code. The `create_precode` function returns two options. Both the LDPC and HDPC codes can be `None`, which means no LDPC or HDPC code is used. If not `None`, the LDPC code is an instance of the LDPC trait, and the HDPC code is an instance of the HDPC trait.

3.2.5 Decoding Configuration

The `CodeConfig` struct stores the decoding configuration:

- `max_inactive_num`: the maximum number of inactive vectors,
- `num_padding`: the number of padding vectors,
- `pre_inactivation`: the type of pre-inactivation,
- `inac_strategy`: the strategy of choosing the inactive variable vectors during dynamic inactivation, and
- `subs_method`: the method of back substitution during the GE phase.

3.3 Precode Traits

3.3.1 LDPC Precode

The LDPC code is defined by the parity-check matrix

$$\begin{bmatrix} S_a & I & S_i \end{bmatrix}$$

where I is the $l \times l$ identity matrix, S_a is the $l \times a$ active parity-check matrix, and S_i is the $l \times (b+h)$ inactive parity-check matrix. All the variable vectors must satisfy the parity-check constraints. While S_a and S_i do not have to be sparse, they must be binary matrices.

The LDPC trait specifies four functions to define the parity-check matrix.

- The `active_column` function returns the indices of the non-zero entries of a column of S_a .

- The `active_row` function returns the indices of the non-zero entries of a row of S_a .
- The `inactive_column` function returns the indices of the non-zero entries of a column of S_i .
- The `inactive_row` function returns the indices of the non-zero entries of a row of S_i .

```

1 fn active_column(&self, var_col: usize) -> Vec<usize>;
2 fn active_row(&self, check_row: usize) -> Vec<usize>;
3 fn inactive_column(&self, var_col: usize) -> Vec<usize>;
4 fn inactive_row(&self, check_row: usize) -> Vec<usize>;

```

3.3.2 HDPC Precode

The finite field used for HDPC-related LU elimination and GF(256) scalar operations on the data manager is selected by the `gf_poly` function of the HDPC trait (not by `CodeScheme`):

```

1 fn gf_poly(&self) -> u16;

```

Return value	HDPC matrix multiply	DataManager
GF2_FIELD_POLY (0x001, sentinel)	XOR only (binary HDPC)	None (GF(2))
A whitelisted degree-8 primitive poly (default 0x11D)	GF(256), e.g. α -scaling	Some (GF256)

Before HDPC precoding (and when building the system solver), the engine calls `manager.config_finite_field(hdpc.gf_poly())`, which also propagates the choice to an attached `DataOperator` when present.

For HDPCs over GF(256), the `gf_poly` function must return one of the sixteen monic primitive polynomials for which $x = 2$ is a primitive element—these are enumerated in `PRIMITIVE_POLYNOMIALS` (see `fountain_engine::algebra::finite_field`). The default is 0x11D, representing $x^8 + x^4 + x^3 + x^2 + 1$. Any polynomial that is either not primitive, or does not produce $x = 2$ as a primitive element—such as AES’s 0x11B—will be rejected by `=GF256::try_new_with_primitive_polynomial`. This check ensures proper field operations and full multiplicative order in GF(256).

```

1 pub const PRIMITIVE_POLYNOMIALS: [u16; 16];
2 pub const GF2_FIELD_POLY: u16 = 0x001;

```

The HDPC code is specified by the parity-check matrix

$$\begin{bmatrix} D & I \end{bmatrix}$$

where D is the $h \times (k+l)$ generator matrix, and I is the $h \times h$ identity matrix. All the variable vectors must satisfy the parity-check constraints. Here D is a dense matrix, and may not be binary. The matrix D can be written as

$$D = [D_a \ D_s \ D_b]$$

where D_a is an $h \times a$ matrix, D_s is an $h \times l$ matrix, and D_b is an $h \times b$ matrix.

To implement the HDPC precode, the HDPC trait specifies three functions: `mul_binary`, `mul_sparse_sh`, and `mul_data`.

The `mul_binary` function implements the multiplication DX for the case that X is a binary matrix. The `mul_binary` function takes the following arguments:

- `gf`: `manager.gf256()` after `config_finite_field` (None for GF(2)-only HDPC); binary implementors may ignore it.
- `params`: the coding parameters,
- `n`: the number of columns of X , which is the number of inactive variables.
- `v`: a function that returns a row of X specified by the input argument.

The output is a $h \times n$ matrix DX .

```

1 fn mul_binary(&self, gf: Option<&GF256>,
2   params: &CodeParams,
3   n: usize,
4   v: &dyn Fn(usize) -> Vec<u8>) -> Vec<Vec<u8>>;

```

The `mul_sparse_sh` function implements the multiplication $D_s S_h$, where S_h is the last h columns of S_i . The `mul_sparse_sh` function takes the following arguments:

- **params**: the coding parameters,
- **s**: a function that returns the indices of the non-zero entries of a row of S_h .

The output is a $h \times h$ matrix $D_s S_h$.

```

1 fn mul_sparse_sh(&self, gf: Option<&GF256>,
2   params: &CodeParams,
3   s: &dyn Fn(usize) -> Vec<usize>) -> Vec<Vec<u8>>;

```

The `mul_data` function implements the multiplication DX for the case that X is a matrix formed by data vectors. As this function is applied to the data vectors, it uses the data manager and the data vector IDs to perform the operations. The `mul_data` function takes the following arguments:

- **manager**: the data manager, which will be introduced in Section 4.
- **params**: the coding parameters.
- **x_ids**: the indices of the vectors forming the matrix X . When the length of **x_ids** is less than $k + l$, the extra rows are assumed to be all-zero vectors.
- **y_ids**: the indices of the vectors forming the matrix $Y = DX$.

```

1 fn mul_data(&self,
2   manager: &mut DataManager,
3   params: &CodeParams,
4   x_ids: &[usize],
5   y_ids: &[usize]);

```

4 Data Manager

The fountain engine specifies **what** tasks the encoder and decoder must accomplish (for example, that a coded vector should be the linear combination of certain message vectors), but it does not dictate **how** these tasks are executed. To model these actions abstractly, the fountain engine defines a set

of operations, each of which includes a name and a list of data vector IDs it involves.

The encoder and decoder use the data manager to manage the data vector IDs and the associated operations, which can be accessed by the `manager` field of the encoder/decoder. In practice, users only need to interact with the data manager when implementing the HDPC precode. However, understanding how data vector IDs are allocated, as well as how the data manager records operations, is important for using the engine correctly and efficiently.

4.1 Types of Vectors

The engine uses data vector IDs to refer to the data vectors. The data vector IDs range is shared by message vectors, variable vectors generated by precoding, and coded vectors generated by the LT encoding.

4.1.1 Message Vectors

Message vectors are used as the encoding input and decoding output, and are indexed from 0 to $k - 1$, where k is the number of message vectors. The following rules of the data vector IDs for message vectors are used:

- Before encoding, the message vectors have data vector IDs from 0 to $k - 1$.
- After decoding, the decoded message vectors have data vector IDs from 0 to $k - 1$.

Due to this rule, data vector IDs for message vectors are not explicitly shown in the encoder and decoder interfaces.

4.1.2 Variable Vectors

Variable vectors are generated by precoding, and are indexed from 0 to $k + l + h - 1$, where $k + l + h$ is the total number of variable vectors. The first $a + l$ variable vectors are the active variable vectors, and the remaining $b + h$ variable vectors are the inactive variable vectors.

The data manager maintains a consecutive range of data vector IDs for the $k + l + h$ variable vectors. By knowing the initial data vector ID for the variable vector 0, all the other data vector IDs for the variable vectors can be determined.

The data vector IDs for the variable vectors are assigned according to the code type:

- For ordinary code `CodeType::Ordinary`, the data vector IDs for the variable vectors are from 0 to $k + l + h - 1$.
- For systematic code `CodeType::Systematic`, the data vector IDs for the variable vectors are from k to $2k + l + h - 1$.

4.1.3 Coded Vectors

Coded vectors include the message vectors for systematic encoding, LDPC and HDPC constraints and LT encoded vectors. We adopt the following convention of coded vector IDs:

- IDs from 0 to $k - 1$ are for message vectors, which are the first k coded vectors for systematic encoding. Ordinary encoding does not use these coded vector IDs.
- IDs from k to $k + l - 1$ are for LDPC constraints.
- IDs from $k + l$ to $k + l + h - 1$ are for HDPC constraints.
- IDs from $k + l + h$ are for LT encoded coded vectors.

The maximum coded vector ID should be known from the design.

LDPC and HDPC constraints are always the all-zero vectors and hence do not need to be transmitted. The other coded vector IDs should be accompanied with the coded IDs for transmission.

4.2 Usage of Coded Vector IDs

4.2.1 Encoding

Now we can explain the input and output of the `encode_coded_vector` function of `Encoder` struct with more details:

- if `coded_id < k` and the code type is `CodeType::Systematic`, the encoder returns `Some(coded_id)`, which is the data vector ID for the message vector with ID `coded_id`.

- if `coded_id < k` and the code type is `CodeType::Ordinary`, the encoder returns `None`, as message vectors are not used as coded vectors in ordinary encoding.
- if `k <= coded_id < k+l+h`, the encoder returns `None`, as LDPC and HDPC constraints are not encoded as separate coded vectors (they are always all-zero vectors).
- if `coded_id >= k+l+h`, the encoder returns `Some(data_id)`, where `data_id` is the data vector ID for the LT encoded coded vector with ID `coded_id`.

4.2.2 Decoding

For systematic decoding, the input coded vector IDs can be either less than `k` or greater than or equal to `k+l+h`. For ordinary decoding, the input coded vector IDs should be greater than or equal to `k+l+h`. The coded IDs from `k` to `k+l+h-1` are used internally by the decoder for LDPC and HDPC constraints, and should not be used when calling the `add_coded_vector` function and the `add_coded_id` function of `Decoder` struct.

4.3 Management of Data IDs

The data manager maintains many functions to manage the data vector IDs. These functions are most used internally by the engine, and are not needed by the users.

For encoding a coded vector with ID `coded_id`, in the `encode_coded_vector` function of `Encoder` struct, the `coded_data_id` function of the data manager is used to get the data vector ID for the coded vector. In addition to assigning a data vector ID for the coded vector, the `coded_data_id` function also performs a data vector operation to ensure that the data vector is an all-zero vector.

For decoding a coded vector with ID `coded_id`, in the `add_coded_id` function of `Decoder` struct, the `insert_coded_id` function of the data manager is used to get the data vector ID for the coded vector. The `insert_coded_id` function does not perform any data vector operation.

One data ID function that may be used by the users is the `temp_data_id` function. This function is used by the engine to get a temporary data vector ID for a data vector generated by the engine, but not used by the users

directly. The `temp_data_id` function may be used in the implementation of the HDPC trait `mul_data` function.

4.4 Finite Field Configuration

The data manager keeps an optional GF(256) instance for scalar multiply/divide and for consistency:

```

1 pub fn config_finite_field(&mut self, pp: u16);
2 pub fn config_finite_field_from(&mut self, gf: &GF256);
3 pub fn gf256(&self) -> Option<&GF256>;

```

- `config_finite_field(pp)`: if `pp = GF2FIELDPOLY`, clears GF(256) (GF(2) mode); otherwise builds tables for `pp`.
- `config_finite_field_from(gf)`: clones an existing GF256 (avoids rebuilding tables).
- Both functions forward the configuration to the attached `DataOperator` when one is set.

In GF(2) mode, `multiply_scalar` and `divide_scalar` panic if a non-trivial scalar is requested (pivot 1 is a no-op).

4.5 Data Operation Interfaces

The encoder and decoder perform the data vector operations through the interface provided by the data manager. The data manager does not implement these operations, but saves them. The saved operations can be obtained by the `get_operations` function of the data manager. We discuss these operations provided by the data manager in the following two categories:

- Data vector operations, and
- Informative operations.

4.5.1 Data Vector Operations

The following data vector operations are supported by the data manager:

- `ensure_zero(list_id)`: ensure the vectors indexed in `list_id` are all zero vectors.
- `add_to_vector(list_id,id)`: add the vectors indexed in `list_id` to the vector `id`.
- `broadcast_add(id,list_id)`: add a vector `id` to all the vectors in `list_id`.
- `multiply_alpha(id)`: multiply the vector `id` by the primitive element α .
- `multiply_scalar(scalar,id)`: multiply the vector `id` by an 8-bit scalar, represent an element in GF(256).
- `divide_scalar(scalar,id)`: divide the vector `id` by an 8-bit scalar, represent an element in GF(256).
- `multiply_add(src_id,scalar,target_id)`: multiply the vector `src_id` by an 8-bit scalar, represent an element in GF(256), and add the result to the vector `target_id`.
- `move_to(src_id,target_id)`: move the data vector with `src_id` to the vector with `target_id`. After moving, the original data vector becomes an all-zero vector.
- `copy_to(src_id,target_id)`: copy the vector with `src_id` to the vector with `target_id`.
- `remove(id)`: remove the vector with `id`.

Users of the engine do not need to directly use these operations. For code scheme implementation, the `mul_data` function in the HDPC trait uses the data operation interfaces.

4.5.2 Informative Operation

The data manager also provides one informative operation:

- `InfoCodedVector(coded_id,data_id)`: the data vector with `data_id` corresponds to the coded vector with `coded_id`.

For encoding, the `InfoCodedVector` operation is used to tell the data operator that a coded vector has been generated. For decoding, the `InfoCodedVector` operation is used to tell the data operator that a coded vector should be added.

The data manager provides two functions that generate this operation:

- `add_coded_vector(coded_id,data_id)`: used for decoding to add a coded vector.
- `encode_coded_vector(coded_id,data_id)`: used for encoding to indicate a coded vector has been generated.

Both functions create the same `InfoCodedVector` operation.

4.5.3 Extract Operations

For an encoder or a decoder, the operations generated can be extracted via the data manager in the `manager` field, which provides the following functions for managing the operations:

- `get_operations`: return a reference to the saved operations by the data manager.
- `clear_operations`: clear all the saved operations.
- `move_new_operations`: return a vector of the new operations generated since the last extraction, but do not change the operations saved in the data manager.

```

1 impl DataManager {
2     pub fn get_operations(&self) -> &[Operation];
3     pub fn clear_operations(&mut self);
4     pub fn move_new_operations(&mut self) -> Vec<Operation>;
5 }

```

5 Data Operators

Various data operators can be defined for the engine. We will discuss the implementation of a data operator according to several typical use cases. Before discussing the implementation of a data operator, we first introduce the `Operation` enum, which is required by all data operators.

5.1 Operation Enum

The `Operation` enum is defined as follows:

```

1 pub enum Operation {
2     EnsureZero { list_id: Vec<usize> },
3     MultiplyAlpha { id: usize },
4     MultiplyScalar { scalar: u8, id: usize },
5     AddToVector { list_id: Vec<usize>, target_id: usize },
6     BroadcastAdd { src_id: usize, target_ids: Vec<usize> },
7     MulAdd { src_id: usize, scalar: u8, target_id: usize },
8     MoveTo { src_id: usize, target_id: usize },
9     CopyTo { src_id: usize, target_id: usize },
10    Remove { id: usize },
11    InfoCodedVector { coded_id: usize, data_id: usize },
12 }

```

All these operations correspond to the data vector operations provided by the data manager. Note that though the data manager provides the `divide_scalar` operation, it is not included in the `Operation` enum. This is because the division by a scalar is converted to other operations internally by the data manager.

5.2 Delayed Data Operation

Following the basic use case in Section 2, we can implement a data operator to execute the operations generated by the engine. For this purpose, the data operator does not need to follow any specific interface, but only needs to execute the operations generated by the engine.

For delayed execution, the data operator can implement any interface to execute the operations. If implementing the `DataOperator` trait (see Section

5), the `execute` function takes a single `&Operation`, so operations need to be executed one at a time. For example, a data operator might have:

- `insert_vector(vector: &[u8], data_id: usize)`: insert the data vector `vector` into the data operator with data ID `data_id`.
- `execute(op: &Operation)`: execute a single operation `op`.

Note that when executing multiple operations, you need to iterate over them and call `execute` for each one.

5.2.1 Encoding Operation

Before executing any operations, the `data_operator` should prepare the message vectors for encoding, which are the first `k` data vectors. The preparation of the message vectors is not included in the operations extracted from the encoder, but is assumed by the encoder. Initialization of the message vectors can be executed even after calling the encoder, but should be before calling the `execute` function of the data operator.

```

1 for i in 0..k {
2     data_operator.insert_vector(&message_vectors[i], i);
3 }

```

After creating the encoder by the `new` function, the operations of precoding are already generated and can be executed by the data operator.

```

1 let encoder = Encoder::new(code_scheme);
2 let operations = encoder.manager.move_new_operations();
3 for op in operations {
4     data_operator.execute(&op);
5 }

```

To perform encoding, the `encode_coded_vector` function is called for each desired coded vector ID. The function `move_new_operations` of the data manager can be called to retrieve the new operations generated since the last extraction, which can be passed to the data operator for execution.

You have flexibility in how you interleave encoding and execution. For example, you may process operations one vector at a time:

```

1 for coded_id in coded_ids {
2     encoder.encode_coded_vector(coded_id);
3     let operations = encoder.manager.move_new_operations();
4     for op in operations {
5         data_operator.execute(&op);
6     }
7 }

```

Alternatively, you may encode all desired vectors first, and then execute all pending operations together:

```

1 let encoder = Encoder::new(code_scheme);
2 for coded_id in coded_ids {
3     encoder.encode_coded_vector(coded_id);
4 }
5 for i in 0..k {
6     data_operator.insert_vector(&message_vectors[i], i);
7 }
8 let operations = encoder.manager.get_operations();
9 // optionally clear the operations after extraction
10 encoder.manager.clear_operations();
11 for op in operations {
12     data_operator.execute(op);
13 }

```

The informational operation `InfoCodedVector` tells the data operator that a coded vector has been generated.

5.2.2 Decoding Operation

After creating the decoder by the `new` function, the operations of initializing some all-zero vectors are generated and can be executed by the data operator.

```

1 let decoder = Decoder::new(code_scheme);
2 let operations = decoder.manager.move_new_operations();
3 for op in operations {

```

```

4     data_operator.execute(&op);
5 }

```

To perform decoding, a sequence of coded vectors are added to the decoder by the `add_coded_id` function. The operations of adding the coded vectors can be retrieved by the `move_new_operations` function of the data manager and executed by the data operator.

```

1  for coded_id in coded_ids {
2      let status = decoder.add_coded_id(coded_id);
3      let operations = decoder.manager.move_new_operations();
4      for op in operations {
5          data_operator.execute(&op);
6      }
7      if status == DecodeStatus::Decoded {
8          break;
9      }
10 }

```

Alternatively, all the operations can be retrieved by the `manager.get_operations` function of the `manager` field and executed by the data operator after the decoding is complete.

```

1  let decoder = Decoder::new(code_scheme);
2  for coded_id in coded_ids {
3      let status = decoder.add_coded_id(coded_id);
4      if status == DecodeStatus::Decoded {
5          break;
6      }
7  }
8  let operations = decoder.manager.get_operations();
9  // optionally clear the operations after extraction
10 decoder.manager.clear_operations();
11 for op in operations {
12     data_operator.execute(op);
13 }

```

It is not necessary to explicitly call the `insert_vector` function to add coded vectors to the data operator, as the `operations` retrieved by the `get_operations` function of the `manager` field already contain the informative operations `InfoCodedVector` to tell the data operator when a coded vector should be added.

5.2.3 Optimization of Data Operation Execution

Though the operations are generated in order, they are not necessary to be executed in the same order. Each operation has one or multiple data ID affected, and some data IDs used (no matter affected or not). We have the following rules:

- Two operations A and B must be executed in the same order as they are generated if a data ID affected by one operation is referred to by another operation.
- A consecutive sequence of operations can be executed in parallel (or any order) if any data ID affected by an operation is not used by all other operations in the sequence.

In delayed data operation execution, the operations retrieved can be optimized before execution. How to optimize the execution depends on the specific data operator implementation, and is not discussed in this guide.

5.3 On-the-fly Data Operation Retrieval

The engine also provides an interface for on-the-fly retrieval and execution of the operations. For this purpose, the data operator needs to implement the `DataOperator` trait, and create encoder and decoder with such a data operator.

```

1 pub trait DataOperator {
2     fn insert_vector(&mut self, vector: &[u8], data_id: usize);
3     fn get_vector(&self, data_id: usize) -> &[u8];
4     fn execute(&mut self, op: &Operation);
5     fn config_finite_field(&mut self, pp: u16) { /* default: no-op */ }
6     fn config_finite_field_from(&mut self, gf: &GF256) {
7         self.config_finite_field(gf.primitive_polynomial());

```

```

8     }
9 }

```

The default implementation of `get_vector` returns an empty slice. Data operators used only for capturing or logging operations (e.g. `DisplayDataOperator`) can rely on this default; operators that store and retrieve vectors must implement `get_vector` to return the stored data. In-memory operators such as `VecDataOperator` should implement `config_finite_field` (and preferably `config_finite_field_from` by cloning `gf`) so that executed operations use the same field as the data manager.

The utility library provides helpers that create a manager and operator with matching fields:

```

1 use fountain_utility::{data_manager_with_vec_operator, data_manager_with_vec_oper
2 let manager = data_manager_with_vec_operator(t, 0x11D);
3 let manager = data_manager_with_vec_operator_from(t, &field);

```

5.3.1 Encoder and Decoder with Data Operator

The encoder and decoder can be created with a data operator by calling the `new_with_operator` function.

```

1 let encoder = Encoder::new_with_operator(
2     code_scheme,
3     Box::new(data_operator));
4 let decoder = Decoder::new_with_operator(
5     code_scheme,
6     Box::new(data_operator));

```

In Rust, the above operation moves the ownership of the data operator into the encoder and decoder. The data operator can be retrieved by the `move_operator` function of the data manager for other data vector operations, and then set back to the data manager for on-the-fly data operation execution.

Note that `move_operator` will panic if no operator has been set. It is safe to call `move_operator` after creating an encoder or decoder with `new_with_operator`.

```

1 let data_operator = encoder.manager.move_operator();
2 encoder.manager.set_operator(data_operator);
3 let data_operator = decoder.manager.move_operator();
4 decoder.manager.set_operator(data_operator);

```

5.3.2 Display Data Operator

Here we illustrate a simple example of data operator that prints the data operations on the console. This data operator only needs to implement the `execute` function of the `DataOperator` trait, and leaves the other functions with the default implementation.

```

1 struct DisplayDataOperator;
2 impl DataOperator for DisplayDataOperator {
3     fn execute(&mut self, op: &Operation) {
4         println!("operation: {:#?}", op);
5     }
6 }

```

In the above example, the `execute` function prints the operation on the console. The other two functions of the `DataOperator` trait use the default implementation. In other words, this is a trivial data operator that exposes the operations. The exposed operations can be captured and executed by other programs.

After creating the encoder and decoder with the data operator, call the `encode_coded_vector` and `add_coded_id` functions to generate the operations, and the operations will be printed on the console.

```

1 // Encoding
2 for coded_id in coded_ids {
3     encoder.encode_coded_vector(coded_id);
4 }
5 // Decoding
6 for coded_id in coded_ids {
7     decoder.add_coded_id(coded_id);
8 }

```

See Section 6.1.1 for a more flexible IO data operator provided in the utility library.

5.4 On-the-fly Data Operation Execution

Finally, we illustrate how to execute the operations on the fly. This approach is useful when testing the code engine but is not recommended for production use as the user loses the flexibility of executing the operations in the desired order. It is recommended to use the on-the-fly data operation retrieval instead to achieve the same purpose.

A data operator for on-the-fly data operation execution needs to implement the `insert_vector` and `get_vector` functions of the `DataOperator` trait nontrivially. We use the `VecDataOperator` struct as an example, which is provided in the utility library.

Before creating the encoder, initialize the data operator with the message vectors. Then create the encoder with the data operator.

```

1 let data_vector_length = 4; // length of each data vector in bytes
2 let mut data_operator = VecDataOperator::new(data_vector_length);
3 for i in 0..k {
4     data_operator.insert_vector(&message_vectors[i], i);
5 }
6 let encoder = Encoder::new_with_operator(
7     code_scheme,
8     Box::new(data_operator));

```

Then call the `encode_coded_vector` function to generate encoded vectors, which can be retrieved by the `get_data_vector` function of the `manager` field of the encoder.

To decode the encoded vectors, call the `add_coded_vector` function to add the coded vectors to the decoder.

```

1 for coded_id in coded_ids {
2     if let Some(data_id) = encoder.encode_coded_vector(coded_id) {
3         let data_vector = encoder.manager.get_data_vector(data_id);
4         decoder.add_coded_vector(coded_id, data_vector);
5     }
6 }

```

6 Fountain Utility

Accompanying the code engine, we provide the `fountain_utility` library for fountain code performance evaluation and testing. The utility library includes the following modules:

- two data operators: `IoDataOperator` and `VecDataOperator`, and
- a testing module `code_testing` for testing fountain code schemes.
- a statistics module `testing_statistics` for collecting and analyzing statistics from multiple test runs of the same code scheme.

To use the utility library, you need to add it as a dependency to your Rust project.

```
1 [dependencies]
2 fountain_utility = "1.0.0" # change the version number
```

6.1 Data Operators

A couple simple data operators are provided in the `fountain_utility` library for code scheme validation and performance evaluation.

- `IoDataOperator` is a data operator that writes the operations to an I/O stream.
- `VecDataOperator` is a data operator that stores the data vectors in a vector.

6.1.1 IoDataOperator

The `IoDataOperator` is a data operator that writes the operations to an I/O stream. It can be customized to write the operations to a file or a console for further processing. To use the `IoDataOperator`, add the following code to your Rust program:

```
1 use fountain_utility::IoDataOperator;
```

The following code shows how to use the `IoDataOperator` to write the operations to the console, as `DisplayDataOperator`.

```

1 use std::io::stdout;
2
3 let mut io_operator = IoDataOperator::new(stdout());
4 let encoder = Encoder::new_with_operator(code_scheme,
5     Box::new(io_operator));

```

The following code shows how to use the `IoDataOperator` to write the operations to a file:

```

1 use std::fs::File;
2 let file = File::create("operations.log"?);
3 let mut file_logger = IoDataOperator::new(file);
4 let encoder = Encoder::new_with_operator(code_scheme,
5     Box::new(file_logger));

```

The following code shows how to use the `IoDataOperator` to write the operations to a memory buffer:

```

1 use std::io::Cursor;
2 let mut buffer = Vec::new();
3 let mut test_logger = IoDataOperator::new(Cursor::new(&mut buffer));
4 let encoder = Encoder::new_with_operator(code_scheme,
5     Box::new(test_logger));

```

6.1.2 VecDataOperator

The `VecDataOperator` is a data operator that stores the data vectors in a vector, and performs the data vector operations on the vectors. It is used for testing the correctness of the code engine and a code scheme for data vector operations. See Section 5.4 for an example of using the `VecDataOperator`.

Each `VecDataOperator` holds its own GF256 tables for `multiply_scalar`, `mul_add`, and `multiply_alpha`. When used with `Encoder::new_with_operator` / `Decoder::new_with_operator`, the field is synchronized automatically

when the engine calls `config_finite_field` during precoding or system-solver setup. For standalone tests, use `data_manager_with_vec_operator` or `data_manager_with_vec_operator_from` so the manager and operator stay aligned.

6.2 Performance Metrics

The basic performance metrics for a code scheme include

1. the encoding and decoding computational costs,
2. the encoding and decoding storage costs, and
3. the decoding success probability or overhead.

The overall computational cost includes the execution time of the fountain code engine and the data operation execution time. The former can be directed measured when executing the engine. Here we mainly care about the data operation computational costs. Instead of directly measuring the data operation execution time during execution, we count the operations transferred to the data manager, which provides an implementation-independent way to measure the computational costs.

6.2.1 PerformanceMetrics Struct

The performance metrics can be counted by the `PerformanceMetrics` struct in the `fountain_utility` library. The structure provides the following fields:

- `multiply_alpha`: the number of scalar-vector multiplication operations, where the scalar is the primitive element α .
- `multiply_scalar`: the number of scalar-vector multiplication operations, where the scalar can be any 8-bit scalar.
- `vector_add`: the number of vector addition operations.
- `mul_add`: the number of operations that multiply a vector by an 8-bit scalar, and add the result to another vector.
- `max_storage`: the maximum number of data vectors stored in the data operator.

- `num_coded_vectors`: the number of coded vectors generated or added to the decoder.

These fields can be counted by the `from_operations` function of the `PerformanceMetrics` struct.

```
1 let metrics = PerformanceMetrics::from_operations(&ops);
```

In the following, we discuss how the performance of a code scheme is evaluated by the code engine.

6.2.2 Computational Costs

We classify the computational costs into the following categories: `multiply_alpha`, `multiply_scalar`, `vector_add`, and `mul_add`. The following operations in the `Operation` enum involve the computational costs:

- `MultiplyAlpha(id)`: one `multiply_alpha` operation.
- `MultiplyScalar(scalar,id)`: one `multiply_scalar` operation.
- `AddToVector(list_id,target_id)`: size of `list_id` number of `vector_add` operations.
- `BroadcastAdd(src_id,target_ids)`: size of `target_ids` number of `vector_add` operations.
- `MulAdd(src_id,scalar,target_id)`: one `mul_add` operation.

Here we count scalar division the same as scalar-vector multiplication, as the former can be implemented by first inverting the scalar and then calling scalar-vector multiplication. Actually, to reduce the complexity of implementing the data operations, scalar division should not be used. Only the code engine performs scalar division.

Scalar-vector multiplication with a general scalar and α , the primitive element, are counted separately as the latter is a special case of the former that can be implemented simpler. Some operations of HDPC are designed specifically using scalar-vector multiplication with α .

One `MulAdd` operation can be counted as one `multiply_scalar` operation and one `vector_add` operation, but that is not the optimal implementation. This approach needs two passes of the vector. A better one is to implement `MulAdd` as a single operation so that one pass is enough.

6.2.3 Decoding Overhead

The minimum number of LT coded vectors required for decoding is k . The decoding overhead o is the number of LT coded vectors used for decoding subtracting k .

The decoding overhead can be derived based on the informative operation `InfoCodedVector` generated by the decoder. The total number of `InfoCodedVector` operations is counted as the `num_coded_vectors` field of the `PerformanceMetrics` struct, from which subtracting k gives the decoding overhead.

6.2.4 Storage Costs

The following operations in the `Operation` enum affect the storage:

- `EnsureZero(list_id)`: initialize and set the list of vectors to all-zero, and generate `list_id` number of new data vectors.
- `MoveTo(src_id,target_id)`: move the vector from the source data vector ID to the target data vector ID, and do not generate any new data vectors.
- `CopyTo(src_id,target_id)`: copy the vector from the source data vector ID to the target data vector ID. It generates one new data vector.
- `Remove(id)`: remove the vector with the data vector ID, and do not generate any new data vectors. It removes one data vector.

The maximum number of data vectors stored in the data operator is counted as the `max_storage` field of the `PerformanceMetrics` struct.

We discuss the storage costs for different types of encoding and decoding, and show that the storage costs can be derived from the coding parameters and the decoding overhead. This discussion can help us to verify the storage cost calculation of a code scheme.

For ordinary encoding, the k message vectors must be stored in the data operator, and they are used to generate the extra $l + h$ variable vectors. Note that during the HDPC calculation, one temporary data vector is used, which is removed after the HDPC calculation. Therefore, after precoding,

the storage costs are $k + l + h$ vectors. The storage cost for each coded vector generated is 1 vector.

For systematic encoding, the k message vectors must be stored in the data operator, and they are duplicated for the $k + l + h$ variable vectors. Therefore, after precoding, the storage costs are $2k + l + h$ vectors. However, the duplication operations `CopyTo` of the message vectors can be executed as `MoveTo` in a data operator if the message vectors are not used anymore. For this implementation, the storage costs are $k + l + h$ vectors.

For ordinary decoding, l LDPC coded vectors are added to the decoder initially, and then LT encoded coded vectors are added one by one. The minimum number of LT coded vectors required for decoding is k . If decoding is successful in the BP phase, the storage cost is $k + l + o$ vectors, where o is the decoding overhead. If decoding goes into the GE phase, h HDPC coded vectors are added to the decoder, and then the storage cost is $k + l + h + o$ vectors.

For systematic decoding, if an LT coded vector has coded ID less than k , it is the original message vector, and the coded vector is duplicated. Therefore, the storage cost is $2k + l + h + o$ vectors.

6.3 Code Testing

6.3.1 Testing Templates

The utility library provides a testing module `code_testing` for testing fountain code schemes. The module provides two testing templates and a helper function:

- `test_code_scheme`: tests a code scheme without actual data vector operations.
- `test_code_scheme_with_data_vectors`: tests a code scheme with actual data vector operations, using the `VecDataOperater` data operator.
- `test_code_scheme_multiple`: runs multiple tests and collects the results.

```

1 pub fn test_code_scheme<C>(
2     code_scheme: &C,
3     k: usize,
```

```

4     num_coded_vectors: usize
5     ) -> TestResult
6 where
7     C: CodeScheme + Clone;
8
9 pub fn test_code_scheme_with_data_vectors<C>(
10     code_scheme: &C,
11     k: usize,
12     data_vector_length: usize,
13     num_coded_vectors: usize
14     ) -> TestResult
15 where
16     C: CodeScheme + Clone;
17
18 pub fn test_code_scheme_multiple<C>(
19     num_runs: usize,
20     code_scheme: &C,
21     k: usize,
22     num_coded_vectors: usize
23     ) -> Vec<TestResult>
24 where
25     C: CodeScheme + Clone;

```

Both test templates return a **TestResult** struct that contains the performance metrics and the test result. The **TestResult** struct provides the following fields:

- **k**: the number of source symbols.
- **num_mismatches**: the number of mismatches between decoded vectors and message vectors.
- **precoding_metrics**: the performance metrics of the precoding operations.
- **encoding_metrics**: the performance metrics of the encoding operations.
- **decoding_metrics**: the performance metrics of the decoding operations.

- `precoding_time_ms`: the time taken for precoding in milliseconds.
- `encoding_time_ms`: the time taken for encoding in milliseconds.
- `decoding_time_ms`: the time taken for decoding in milliseconds.

Test results can be saved to and loaded from files using JSON Lines format:

```

1 // Save results to a file
2 save_test_results(&results, "results.jsonl"?;
3
4 // Load results from a file
5 let loaded_results = load_test_results("results.jsonl"?;

```

6.3.2 Testing Statistics

A coding scheme needs to be tested multiple times to get a reliable result. As packet loss is simulated by randomly shuffling the coded vectors, the test result is not the same. The following code shows how to test a code scheme multiple times and collect the testing results.

```

1 let results = test_code_scheme_multiple(
2     num_runs,
3     &code_scheme,
4     k,
5     num_coded_vectors);

```

Alternatively, you can manually collect results:

```

1 let results: Vec<TestResult> = (0..num_runs).map(|_i| {
2     test_code_scheme(&code_scheme, k, num_coded_vectors)
3 }).collect();

```

All the testing results have the same value of `k` and `num_coded_vectors`. The `TestStatistics` struct provides static methods to get the statistics from the test results:

- `success_rate(results)`: returns `(num_runs, successful_runs, success_rate)` tuple.
- `overhead_stats(k, results)`: returns `Statistics` with mean, min, and max of the decoding overhead.
- `storage_stats(results)`: returns `Statistics` with mean, min, and max of the storage costs.
- `avg_computation_costs(k, results)`: returns a tuple of `(precoding_avg_comp, encoding_avg_comp, decoding_avg_comp)` with average computational costs.
- `avg_time_costs(results)`: returns `AverageTime` with average time taken for precoding, encoding, and decoding when executing the code engine.

References

- [1] M. Luby, “LT codes,” in *Foundations of Computer Science (FOCS), 2002. Proceedings. The 43rd Annual IEEE Symposium on*, Nov. 2002, pp. 271–280.
- [2] A. Shokrollahi, “Raptor codes,” *Information Theory, IEEE Transactions on*, vol. 52, no. 6, pp. 2551–2567, June 2006.
- [3] A. Shokrollahi, T. Stockhammer, M. Luby, and M. Watson, “Raptor Forward Error Correction Scheme for Object Delivery,” RFC 5053, Oct. 2007. [Online]. Available: <https://www.rfc-editor.org/info/rfc5053>
- [4] L. Minder, A. Shokrollahi, M. Watson, M. Luby, and T. Stockhammer, “RaptorQ Forward Error Correction Scheme for Object Delivery,” RFC 6330, Aug. 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6330>
- [5] TON Foundation, “Reliable large datagram protocol (rldp),” TON Documentation, 2024, tON blockchain network protocol documentation. [Online]. Available: <https://docs.ton.org/v3/documentation/network/protocols/rldp>

-
- [6] Libreicast Team, “Libreicast project,” Open Source Project, 2024, IPv6 multicast implementation based on RaptorQ. [Online]. Available: <https://codeberg.org/libreicast>
 - [7] L. Fulchir, “libraptorq: C++11 implementation of rfc6330,” Open Source Project, 2016, header-only C++11 library implementing RFC6330 RaptorQ with C/C++98 API support. Features deterministic builds, LZ4 compression for cached pre-computations, and multi-language bindings. [Online]. Available: <https://github.com/LucaFulchir/libRaptorQ>
 - [8] OpenRQ Team, “Openrq: Java implementation of raptorq,” Open Source Project, 2024, java implementation of RaptorQ forward error correction. [Online]. Available: <https://openrq-team.github.io/openrq/>
 - [9] Y. Poirier, “A rust library for implementing forward error correction (fec) using raptor codes,” Crates.io Package, 2025, rust implementation of RFC5053 Raptor 10 forward error correction. [Online]. Available: <https://crates.io/crates/raptor-code>
 - [10] C. Berner, “Raptorq: Rust implementation,” Crates.io Package, 2024, rust implementation of RFC6330 RaptorQ forward error correction. [Online]. Available: <https://crates.io/crates/raptorq>